Redux Toolkit + Axios

Complete Step-by-Step Guide

Building a Shopping Cart App with React, Redux Toolkit, and Axios

Redux Toolkit + Axios -- Complete Step-by-Step Guide

Building a shopping cart app with React, Redux Toolkit, and Axios -- from an empty folder to a live product catalog fetched from a real API, with a fully working cart.

What you'll have by the end: a product grid pulled live from an API via axios, Add to Cart with quantity merging, a dedicated cart view with +/- controls and remove, a running total, a cart that survives a page reload via localStorage, a full checkout flow with a shipping form and order confirmation, and a UI that holds up on a phone screen.

Folder structure you're building toward:

```
redux-cart-app/
|---- src/
|   |---- api/
|   |   `---- axiosClient.js
|   |---- app/
|   |   |---- store.js
|   |   `---- localStorage.js
|   |---- features/
|   |   |---- cartSlice.js
|   |   `---- productsSlice.js
|   |---- components/
|   |   |---- Navbar.jsx
|   |   |---- ProductCard.jsx
|   |   |---- ProductList.jsx
|   |   |---- Cart.jsx
|   |   |---- Checkout.jsx
|   |   `---- OrderConfirmation.jsx
|   |---- App.jsx
|   |---- App.css
|   |---- index.jsx
|   `---- index.css
|---- index.html
|---- vite.config.js
`---- package.json
```

Note on `index.jsx`, not `index.js`: the entry file contains JSX. Plain Vite only parses JSX syntax in files ending `.jsx`/`.tsx` -- naming it `.js` and trying to coax esbuild into treating it as JSX is more fragile than it looks (an `optimizeDeps` override doesn't reach source files at all). Use `.jsx` from the start.

Part 1 -- Project Setup, Folder Structure, Product Listing

1. Create the project and install dependencies

```
npm create vite@latest redux-cart-app -- --template react
cd redux-cart-app
npm install @reduxjs/toolkit react-redux
```

Why Vite instead of Create React App? CRA is no longer maintained. Vite gives instant dev-server startup and is the current standard for new React projects -- same `src/` conventions you already know.

2. Add the product catalog

A static array keeps this first pass focused on Redux, not data-fetching. (Part 7 replaces this with a live API call -- build the static version first so you understand what's being replaced.)

`src/data/products.js`

```
const products = [
 {
   id: 1,
   name: "Wireless Headphones",
   price: 59.99,
   image: "https://via.placeholder.com/200x200?text=Headphones",
 },
 {
   id: 2,
   name: "Smart Watch",
   price: 129.99,
   image: "https://via.placeholder.com/200x200?text=Smart+Watch",
 },
 {
   id: 3,
   name: "Bluetooth Speaker",
   price: 39.99,
   image: "https://via.placeholder.com/200x200?text=Speaker",
 },
 {
   id: 4,
   name: "Mechanical Keyboard",
   price: 89.99,
   image: "https://via.placeholder.com/200x200?text=Keyboard",
 },
];

export default products;
```

Each product carries an `id` (used as the React `key` and later the cart item identifier), a `name`, a numeric `price`, and an `image` URL.

3. Build the display components

`src/components/ProductCard.jsx`

```
// Renders a single product's info. Receives the product object as a prop
// from ProductList, so this component doesn't know about the whole catalog.
function ProductCard({ product }) {
 return (
   <div className="product-card">
     <img src={product.image} alt={product.name} />
     <h3>{product.name}</h3>
     <p className="price">${product.price.toFixed(2)}</p>
     {/* Inert for now -- Part 3 wires this to Redux's addToCart action. */}
```

```
      <button className="add-btn">Add to Cart</button>
    </div>
  );
}

export default ProductCard;
```

src/components/ProductList.jsx

```
import products from "../data/products";
import ProductCard from "./ProductCard";

function ProductList() {
  return (
    <div className="product-list">
      {products.map((product) => (
        // key must be unique and stable -- React uses it to track which
        // DOM node corresponds to which array item across re-renders.
        <ProductCard key={product.id} product={product} />
      ))}
    </div>
  );
}

export default ProductList;
```

src/components/Navbar.jsx

```
// Cart count is hardcoded to 0 for now -- Part 2/3 pulls it from Redux.
function Navbar() {
  return (
    <nav className="navbar">
      <h1 className="logo">[cart] ReduxCart</h1>
      <div className="cart-icon">
        Cart (<span className="cart-count">0</span>)
      </div>
    </nav>
  );
}

export default Navbar;
```

src/App.jsx

```
import Navbar from "./components/Navbar";
import ProductList from "./components/ProductList";
import "./App.css";

function App() {
  return (
    <div className="app">
      <Navbar />
      <main>
        <h2>Products</h2>
        <ProductList />
      </main>
    </div>
  );
}

export default App;
```

src/index.jsx

```
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App";
import "./index.css";

const root = ReactDOM.createRoot(document.getElementById("root"));

root.render(
 <React.StrictMode>
   <App />
 </React.StrictMode>
);
```

index.html

```
<!DOCTYPE html>
<html lang="en">
 <head>
   <meta charset="UTF-8" />
   <meta name="viewport" content="width=device-width, initial-scale=1.0" />
   <title>Redux Cart App</title>
 </head>
 <body>
   <div id="root"></div>
   <script type="module" src="/src/index.jsx"></script>
 </body>
</html>
```

4. Base styling

src/index.css

```
* {
 margin: 0;
 padding: 0;
 box-sizing: border-box;
}

body {
 font-family: "Segoe UI", Arial, sans-serif;
 background-color: #f4f4f4;
 color: #222;
}
```

src/App.css

```
.navbar {
 display: flex;
 justify-content: space-between;
 align-items: center;
 padding: 16px 32px;
 background-color: #282c34;
 color: white;
}

.cart-icon { font-weight: bold; cursor: pointer; }

main { padding: 24px 32px; }

.product-list {
 display: grid;
```

```
 grid-template-columns: repeat(auto-fill, minmax(220px, 1fr));
 gap: 20px;
 margin-top: 16px;
}

.product-card {
 background: white;
 border-radius: 8px;
 padding: 16px;
 text-align: center;
 box-shadow: 0 2px 6px rgba(0, 0, 0, 0.1);
}

.product-card img { width: 100%; border-radius: 4px; margin-bottom: 12px; }
.price { color: #444; margin: 8px 0; font-weight: bold; }

.add-btn {
 background-color: #282c34;
 color: white;
 border: none;
 padding: 8px 16px;
 border-radius: 4px;
 cursor: pointer;
}

.add-btn:hover { background-color: #444; }
```

5. Run it

```
npm run dev
```

You should see the navbar and a responsive grid of 4 product cards, each with an inert "Add to Cart" button. That's the whole point of Part 1 -- prove the toolchain works before adding state.

Part 2 -- Redux Toolkit Setup (Store, Slice, Provider)

1. The cart slice

`src/features/cartSlice.js`

```
import { createSlice } from "@reduxjs/toolkit";

const initialState = {
 items: [], // each item: { id, name, price, image, quantity }
};

const cartSlice = createSlice({
 name: "cart",
 initialState,
 reducers: {
   addToCart: (state, action) => {
     const product = action.payload;
     const existingItem = state.items.find((item) => item.id === product.id);

     if (existingItem) {
       existingItem.quantity += 1;
     } else {
```

```
      // Redux Toolkit wraps reducers with Immer, so this "mutation"
      // is safe -- it's converted into an immutable update internally.
      state.items.push({ ...product, quantity: 1 });
    }
  },

  removeFromCart: (state, action) => {
    const productId = action.payload;
    state.items = state.items.filter((item) => item.id !== productId);
  },

  increaseQuantity: (state, action) => {
    const productId = action.payload;
    const item = state.items.find((item) => item.id === productId);
    if (item) item.quantity += 1;
  },

  decreaseQuantity: (state, action) => {
    const productId = action.payload;
    const item = state.items.find((item) => item.id === productId);
    if (item && item.quantity > 1) {
      item.quantity -= 1;
    } else {
      state.items = state.items.filter((item) => item.id !== productId);
    }
  },
 },
});

export const { addToCart, removeFromCart, increaseQuantity, decreaseQuantity } =
 cartSlice.actions;

export default cartSlice.reducer;
```

Why the "mutation" is safe: `createSlice` wraps every reducer with Immer. Code that looks like direct mutation (`state.items.push(...)`) actually produces a correctly immutable update behind the scenes -- this is the main reason Redux Toolkit replaced hand-written reducers.

2. The store

`src/app/store.js`

```
import { configureStore } from "@reduxjs/toolkit";
import cartReducer from "../features/cartSlice";

export const store = configureStore({
 reducer: {
   cart: cartReducer, // state shape: state.cart.items
 },
});
```

`configureStore` sets up good defaults out of the box: Redux DevTools support, `redux-thunk` middleware, and a dev-only check that catches accidental state mutations outside reducers.

3. Wrap the app in `<Provider>`

`src/index.jsx` (updated)

```
import React from "react";
import ReactDOM from "react-dom/client";
import { Provider } from "react-redux";
import { store } from "./app/store";
import App from "./App";
import "./index.css";

const root = ReactDOM.createRoot(document.getElementById("root"));

root.render(
 <React.StrictMode>
   {/* Provider makes the store available to every component below it
       via useSelector/useDispatch, without prop-drilling it down. */}
   <Provider store={store}>
     <App />
   </Provider>
 </React.StrictMode>
);
```

Nothing in the UI changes yet -- the "Add to Cart" button is still inert. Open Redux DevTools' **State** tab and confirm you see `{ cart: { items: [] } }`. That confirms the store is wired correctly before you dispatch a single action.

Part 3 -- Add to Cart Functionality

`src/components/ProductCard.jsx` (updated)

```
import { useDispatch } from "react-redux";
import { addToCart } from "../features/cartSlice";

function ProductCard({ product }) {
 const dispatch = useDispatch();

 const handleAddToCart = () => {
   // addToCart(product) builds { type: "cart/addToCart", payload: product }.
   // dispatch() sends it to the store, which runs the cartSlice reducer.
   dispatch(addToCart(product));
 };

 return (
   <div className="product-card">
     <img src={product.image} alt={product.name} />
     <h3>{product.name}</h3>
     <p className="price">${product.price.toFixed(2)}</p>
     <button className="add-btn" onClick={handleAddToCart}>
       Add to Cart
     </button>
   </div>
 );
}

export default ProductCard;
```

`src/components/Navbar.jsx` (updated)

```
import { useSelector } from "react-redux";

function Navbar() {
 // Subscribes this component to state.cart.items -- re-renders whenever it changes.
 const cartItems = useSelector((state) => state.cart.items);
```

```
 // Sum of quantities, not items.length -- 3 units of one product should count as 3.
 const totalCount = cartItems.reduce((sum, item) => sum + item.quantity, 0);

 return (
   <nav className="navbar">
     <h1 className="logo">[cart] ReduxCart</h1>
     <div className="cart-icon">
       Cart (<span className="cart-count">{totalCount}</span>)
     </div>
   </nav>
 );
}

export default Navbar;
```

Click "Add to Cart" on a product twice -- it shouldn't create two rows internally, it should increment that item's `quantity` to 2 (check Redux DevTools' **Diff** tab to see it happen). The navbar count should climb with every click.

Part 4 -- Cart Page + Remove Item

`src/components/Cart.jsx`

```
import { useSelector, useDispatch } from "react-redux";
import { removeFromCart } from "../features/cartSlice";

function Cart() {
 const cartItems = useSelector((state) => state.cart.items);
 const dispatch = useDispatch();

 const handleRemove = (productId) => {
   dispatch(removeFromCart(productId));
 };

 if (cartItems.length === 0) {
   return <p className="empty-cart">Your cart is empty.</p>;
 }

 return (
   <div className="cart">
     {cartItems.map((item) => (
       <div className="cart-item" key={item.id}>
         <img src={item.image} alt={item.name} />
         <div className="cart-item-info">
           <h3>{item.name}</h3>
           <p>${item.price.toFixed(2)} each</p>
           {/* Quantity is read-only for now -- Part 5 adds +/- controls. */}
           <p>Quantity: {item.quantity}</p>
         </div>
         <button className="remove-btn" onClick={() => handleRemove(item.id)}>
           Remove
         </button>
       </div>
     ))}
   </div>
 );
}

export default Cart;
```

App now owns a small piece of local UI state -- which screen is showing. This is a deliberate line: Redux holds *shared, cross-component* data (the cart), `useState` holds *transient, single-component* state (which page you're looking at).

`src/App.jsx` (updated)

```
import { useState } from "react";
import Navbar from "./components/Navbar";
import ProductList from "./components/ProductList";
import Cart from "./components/Cart";
import "./App.css";

function App() {
 const [view, setView] = useState("products");

 return (
   <div className="app">
     <Navbar onCartClick={() => setView("cart")} />
     <main>
       {view === "products" ? (
         <>
           <h2>Products</h2>
           <ProductList />
         </>
       ) : (
         <>
           <div className="cart-header">
             <h2>Your Cart</h2>
             <button className="back-btn" onClick={() => setView("products")}>
               <- Back to Products
             </button>
           </div>
           <Cart />
         </>
       )}
     </main>
   </div>
 );
}

export default App;
```

`src/components/Navbar.jsx` (updated)

```
import { useSelector } from "react-redux";

function Navbar({ onCartClick }) {
 const cartItems = useSelector((state) => state.cart.items);
 const totalCount = cartItems.reduce((sum, item) => sum + item.quantity, 0);

 return (
   <nav className="navbar">
     <h1 className="logo">[cart] ReduxCart</h1>
     <div className="cart-icon" onClick={onCartClick}>
       Cart (<span className="cart-count">{totalCount}</span>)
     </div>
   </nav>
 );
}

export default Navbar;
```

`src/App.css` (append)

```
.cart-header { display: flex; justify-content: space-between; align-items: center; }

.back-btn {
 background: none;
 border: 1px solid #282c34;
 color: #282c34;
 padding: 6px 14px;
 border-radius: 4px;
 cursor: pointer;
}
.back-btn:hover { background-color: #282c34; color: white; }

.cart { display: flex; flex-direction: column; gap: 16px; margin-top: 16px; }

.cart-item {
 display: flex;
 align-items: center;
 gap: 16px;
 background: white;
 border-radius: 8px;
 padding: 12px 16px;
 box-shadow: 0 2px 6px rgba(0, 0, 0, 0.1);
}
.cart-item img { width: 70px; height: 70px; object-fit: cover; border-radius: 4px; }
.cart-item-info { flex: 1; }

.remove-btn {
 background-color: #d9534f;
 color: white;
 border: none;
 padding: 8px 14px;
 border-radius: 4px;
 cursor: pointer;
}
.remove-btn:hover { background-color: #c9302c; }

.empty-cart { margin-top: 24px; color: #666; }
```

Part 5 -- Quantity Buttons + Total Price

`src/components/Cart.jsx` (updated)

```
import { useSelector, useDispatch } from "react-redux";
import {
 removeFromCart,
 increaseQuantity,
 decreaseQuantity,
} from "../features/cartSlice";

function Cart() {
 const cartItems = useSelector((state) => state.cart.items);
 const dispatch = useDispatch();

 const handleRemove = (id) => dispatch(removeFromCart(id));
 const handleIncrease = (id) => dispatch(increaseQuantity(id));
 const handleDecrease = (id) => dispatch(decreaseQuantity(id));

 // Derived state -- recomputed every render, never stored separately,
 // so it can never drift out of sync with the items array.
 const totalPrice = cartItems.reduce(
   (sum, item) => sum + item.price * item.quantity,
```

```
    0
  );

  if (cartItems.length === 0) {
    return <p className="empty-cart">Your cart is empty.</p>;
  }

  return (
    <div className="cart">
      {cartItems.map((item) => (
        <div className="cart-item" key={item.id}>
          <img src={item.image} alt={item.name} />
          <div className="cart-item-info">
            <h3>{item.name}</h3>
            <p>${item.price.toFixed(2)} each</p>

            <div className="quantity-controls">
              <button className="qty-btn" onClick={() => handleDecrease(item.id)}>-</button>
              <span className="qty-value">{item.quantity}</span>
              <button className="qty-btn" onClick={() => handleIncrease(item.id)}>+</button>
            </div>

            <p className="subtotal">
              Subtotal: ${(item.price * item.quantity).toFixed(2)}
            </p>
          </div>
          <button className="remove-btn" onClick={() => handleRemove(item.id)}>
            Remove
          </button>
        </div>
      ))}

      <div className="cart-total">
        <h3>Total: ${totalPrice.toFixed(2)}</h3>
      </div>
    </div>
  );
}

export default Cart;
```

Why `decreaseQuantity` just works at 1 -> 0: the logic lives in the reducer (Part 2), not the component. When quantity is 1 and you decrease again, the reducer removes the item outright instead of showing "0" -- the component only ever dispatches and trusts the slice to handle the edge case.

`src/App.css` (append)

```
.quantity-controls { display: flex; align-items: center; gap: 10px; margin: 8px 0; }

.qty-btn {
  width: 28px;
  height: 28px;
  border: 1px solid #282c34;
  background: white;
  color: #282c34;
  border-radius: 4px;
  cursor: pointer;
  font-size: 16px;
  line-height: 1;
}
.qty-btn:hover { background-color: #282c34; color: white; }

.qty-value { min-width: 20px; text-align: center; font-weight: bold; }
.subtotal { color: #555; font-size: 0.9em; }
```

```
.cart-total {
 align-self: flex-end;
 margin-top: 8px;
 padding: 16px 24px;
 background: white;
 border-radius: 8px;
 box-shadow: 0 2px 6px rgba(0, 0, 0, 0.1);
}
```

Part 6 -- Final UI Polish + Interview Prep

`src/App.css` (append: responsive)

```
@media (max-width: 600px) {
 .navbar { padding: 12px 16px; }
 main { padding: 16px; }

 .product-list {
   grid-template-columns: repeat(auto-fill, minmax(140px, 1fr));
   gap: 12px;
 }

 .cart-item { flex-wrap: wrap; }
 .cart-item img { width: 56px; height: 56px; }
 .cart-total { align-self: stretch; text-align: right; }
}
```

`src/App.css` (append: micro-interactions)

```
.product-card, .cart-item, .add-btn, .qty-btn, .remove-btn, .back-btn {
 transition: all 0.15s ease-in-out;
}

.product-card:hover {
 transform: translateY(-2px);
 box-shadow: 0 4px 10px rgba(0, 0, 0, 0.15);
}
```

Interview questions this project actually answers

Why Redux Toolkit over plain Redux? Plain Redux needs hand-written action types, action creators, and manually-spread immutable updates. `createSlice` generates the action creators and lets reducers "mutate" state directly via Immer -- that's why `state.items.push(...)` in `cartSlice.js` is safe.

Why does `totalPrice` live nowhere in Redux state? It's fully derivable from `items`. Storing it separately risks it drifting out of sync every time the cart changes -- the standard practice is: store the minimal source of truth, derive the rest at render time.

Why does `view` live in `useState`, not Redux? No other component needs to know which screen is showing. Redux is for state shared across components (the cart contents, touched by `Navbar`, `Cart`, and `ProductCard` alike) -- putting every piece of state in Redux "just in case" is a common overuse mistake.

Why `key={item.id}` and never the array index? React uses `key` to match array items across re-renders. If you key by index and the array order changes (an item removed from the middle), React can misattribute DOM/state to the wrong row. `id` is stable regardless of position.

Part 7 -- Fetching Products with Axios (Redux Async Thunk)

Everything through Part 6 read from the hardcoded `src/data/products.js` array on purpose -- it kept the focus on Redux. This part replaces it with a real network call to the Fake Store API, using Redux Toolkit's `createAsyncThunk` so fetching stays consistent with the cart's Redux-first approach.

```
npm install axios
```

src/api/axiosClient.js

```
import axios from "axios";

// One configured instance, reused everywhere -- if the API base URL ever
// changes, you edit this line instead of every call site.
const axiosClient = axios.create({
 baseURL: "https://fakestoreapi.com",
 timeout: 8000, // fail fast instead of hanging on a dead network
});

export default axiosClient;
```

src/features/productsSlice.js

```
import { createSlice, createAsyncThunk } from "@reduxjs/toolkit";
import axiosClient from "../api/axiosClient";

// Generates an action creator that dispatches pending/fulfilled/rejected
// automatically around the async function -- no manual try/catch needed.
export const fetchProducts = createAsyncThunk(
 "products/fetchProducts",
 async () => {
   const response = await axiosClient.get("/products");
   // Fake Store API returns { id, title, price, image, ... }.
   // Map title -> name so ProductCard/cartSlice keep using `name`.
   return response.data.map((item) => ({
     id: item.id,
     name: item.title,
     price: item.price,
     image: item.image,
   }));
 }
);

const productsSlice = createSlice({
 name: "products",
 initialState: {
   items: [],
   status: "idle", // "idle" | "loading" | "succeeded" | "failed"
   error: null,
 },
 reducers: {},
 extraReducers: (builder) => {
   builder
     .addCase(fetchProducts.pending, (state) => {
       state.status = "loading";
       state.error = null;
     })
     .addCase(fetchProducts.fulfilled, (state, action) => {
       state.status = "succeeded";
       state.items = action.payload;
```

```
    })
    .addCase(fetchProducts.rejected, (state, action) => {
      state.status = "failed";
      state.error = action.error.message;
    });
 },
});

export default productsSlice.reducer;
```

Why track `status` as a string, not a boolean `loading` flag? A boolean can't distinguish "hasn't fetched yet" from "fetching" from "done" from "failed" -- four real states the UI needs to render differently. `extraReducers` handles the three lifecycle actions `createAsyncThunk` auto-generates, which aren't declared in this slice's own `reducers`.

`src/app/store.js` (updated)

```
import { configureStore } from "@reduxjs/toolkit";
import cartReducer from "../features/cartSlice";
import productsReducer from "../features/productsSlice";

export const store = configureStore({
 reducer: {
   cart: cartReducer,
   products: productsReducer,
 },
});
```

`src/components/ProductList.jsx` (updated)

```
import { useEffect } from "react";
import { useSelector, useDispatch } from "react-redux";
import { fetchProducts } from "../features/productsSlice";
import ProductCard from "./ProductCard";

function ProductList() {
 const dispatch = useDispatch();
 const { items: products, status, error } = useSelector(
   (state) => state.products
 );

 useEffect(() => {
   if (status === "idle") {
     dispatch(fetchProducts());
   }
 }, [status, dispatch]);

 if (status === "loading") {
   return <p className="status-msg">Loading products...</p>;
 }
 if (status === "failed") {
   return <p className="status-msg error">Failed to load products: {error}</p>;
 }

 return (
   <div className="product-list">
     {products.map((product) => (
       <ProductCard key={product.id} product={product} />
     ))}
   </div>
 );
}

export default ProductList;
```

The `status === "idle"` guard inside `useEffect` stops re-fetching every time `ProductList` remounts (e.g. switching back from the cart view) -- once `status` flips to `"succeeded"`, the effect is a no-op. Notice `ProductCard` and `cartSlice` needed **zero** changes: the thunk maps the API response to the exact `{ id, name, price, image }` shape the rest of the app already expected.

`src/App.css` (append)

```
.status-msg { margin-top: 24px; color: #666; font-size: 1.1em; }
.status-msg.error { color: #c9302c; }
```

Try it

```
npm run dev
```

You should briefly see "Loading products...", then a real product grid sourced from `https://fakestoreapi.com/products`. Add to Cart, quantities, totals, and the navbar count all keep working unchanged -- they only ever depended on the product shape, never on where the data came from.

`src/data/products.js` is no longer imported anywhere at this point -- safe to delete, or keep as an offline fallback reference.

Part 8 -- localStorage Persistence

Right now, refreshing the page loses the cart -- it only lives in memory, inside the Redux store. This part makes the cart survive a reload by rehydrating the store from `localStorage` on startup, and writing it back after every change.

1. A small localStorage helper module

`src/app/localStorage.js`

```
const CART_STORAGE_KEY = "redux-cart-app/cart";

// Reads the persisted cart out of localStorage on app startup.
// Returns undefined (not null/[]) on any failure so the caller can fall
// back to the slice's own initialState instead of a broken shape.
export function loadCartState() {
 try {
   const serialized = localStorage.getItem(CART_STORAGE_KEY);
   if (serialized === null) return undefined;

   const parsed = JSON.parse(serialized);
   // Minimal shape check -- a corrupted or stale-schema value shouldn't crash the app.
   if (!parsed || !Array.isArray(parsed.items)) return undefined;

   return parsed;
 } catch (err) {
   console.warn("Could not load cart from localStorage:", err);
   return undefined;
 }
}

// Writes the cart slice to localStorage. Called from store.js's subscribe()
// callback, so it runs after every dispatch that changes cart state.
export function saveCartState(cartState) {
 try {
```

```
    localStorage.setItem(CART_STORAGE_KEY, JSON.stringify(cartState));
  } catch (err) {
    // Quota exceeded, private-browsing restrictions, etc. -- persistence is a
    // nice-to-have, so log and move on rather than breaking the cart itself.
    console.warn("Could not save cart to localStorage:", err);
  }
}
```

Both functions are wrapped in `try/catch` on purpose. `localStorage` can throw in private-browsing modes, when the quota is exceeded, or when the stored JSON doesn't match the shape the app expects (e.g. you changed the cart item schema since the user's last visit) -- none of those should ever crash the cart itself, so every failure path just falls back to an empty cart instead.

2. Rehydrate the store on startup, and persist on every change

`src/app/store.js` (updated)

```
import { configureStore } from "@reduxjs/toolkit";
import cartReducer from "../features/cartSlice";
import productsReducer from "../features/productsSlice";
import { loadCartState, saveCartState } from "./localStorage";

// Rehydrate the cart from localStorage before the store is even created.
// If nothing is stored yet (or it's corrupted), this is undefined and
// Redux Toolkit falls back to cartSlice's own initialState automatically.
const persistedCartState = loadCartState();

// configureStore sets up the store with good defaults out of the box:
// Redux DevTools support, redux-thunk middleware, and a dev-only check
// that catches accidental state mutations outside reducers.
export const store = configureStore({
  reducer: {
    cart: cartReducer,
    products: productsReducer,
  },
  preloadedState: persistedCartState ? { cart: persistedCartState } : undefined,
});

// Persist the cart slice to localStorage after every dispatch that changes it.
// Debounced so a burst of clicks (e.g. holding the quantity "+" button)
// writes to disk once, not once per click.
let saveTimeoutId;
let lastSavedCart = store.getState().cart;

store.subscribe(() => {
  const currentCart = store.getState().cart;
  if (currentCart === lastSavedCart) return; // only the cart slice changing is worth a write
  lastSavedCart = currentCart;

  clearTimeout(saveTimeoutId);
  saveTimeoutId = setTimeout(() => {
    saveCartState(currentCart);
  }, 300);
});
```

Why `preloadedState`, not a `useEffect` that dispatches on mount? `preloadedState` sets the store's *initial* state before any component ever renders -- there's no flash of an empty cart followed by a re-render once a `useEffect` fires. It's the mechanism Redux Toolkit ships specifically for this kind of startup rehydration.

Why compare `currentCart === lastSavedCart` before saving? The store's `subscribe()` callback fires after *every* dispatch to the whole store, including ones that only touch `state.products` (like the `fetchProducts` lifecycle

actions from Part 7). Without this check, fetching the product list would trigger a wasted localStorage write of a cart that hasn't actually changed. Because `cartSlice`'s reducers only produce a new `state.cart` reference when they actually modify it (Immer's guarantee), a plain reference check is enough to detect "did the cart specifically change."

Why debounce the write? Holding down the quantity `+` button can dispatch several `increaseQuantity` actions within a few hundred milliseconds. Writing to `localStorage` on every single one is wasted work; debouncing to 300ms coalesces a burst of rapid changes into a single write, without the user ever noticing a delay.

3. Try it

```
npm run dev
```

Add a couple of items to the cart, then reload the page (a full reload, not React's hot-reload). The cart should come back exactly as you left it. Open your browser's DevTools -> Application -> Local Storage and you'll see a `redux-cart-app/cart` entry holding the serialized `{ items: [...] }`. Removing every item from the cart should update that entry back down to `{ "items": [] }`.

Part 9 -- Checkout Flow

The last piece: a way to actually turn a cart into an order. This part adds a `Checkout` view (order summary + a shipping form with validation) and an `OrderConfirmation` view, and teaches `App.jsx` to route between four views instead of two.

1. Add a `clearCart` action

`src/features/cartSlice.js` (updated -- new reducer only, rest unchanged from Part 2)

```
import { createSlice } from "@reduxjs/toolkit";

const initialState = {
 items: [],
};

const cartSlice = createSlice({
 name: "cart",
 initialState,
 reducers: {
   addToCart: (state, action) => {
     const product = action.payload;
     const existingItem = state.items.find((item) => item.id === product.id);

     if (existingItem) {
       existingItem.quantity += 1;
     } else {
       state.items.push({ ...product, quantity: 1 });
     }
   },

   removeFromCart: (state, action) => {
     const productId = action.payload;
     state.items = state.items.filter((item) => item.id !== productId);
   },

   increaseQuantity: (state, action) => {
     const productId = action.payload;
```

```
      const item = state.items.find((item) => item.id === productId);
      if (item) item.quantity += 1;
    },

    decreaseQuantity: (state, action) => {
      const productId = action.payload;
      const item = state.items.find((item) => item.id === productId);
      if (item && item.quantity > 1) {
        item.quantity -= 1;
      } else {
        state.items = state.items.filter((item) => item.id !== productId);
      }
    },

    // Empties the cart entirely -- dispatched once an order is placed.
    clearCart: (state) => {
      state.items = [];
    },
  },
});

export const {
  addToCart,
  removeFromCart,
  increaseQuantity,
  decreaseQuantity,
  clearCart,
} = cartSlice.actions;

export default cartSlice.reducer;
```

2. The Checkout component

`src/components/Checkout.jsx`

```
import { useState } from "react";
import { useSelector, useDispatch } from "react-redux";
import { clearCart } from "../features/cartSlice";

function Checkout({ onOrderPlaced, onBackToCart }) {
  const cartItems = useSelector((state) => state.cart.items);
  const dispatch = useDispatch();

  const [form, setForm] = useState({
    name: "",
    email: "",
    address: "",
    city: "",
    postalCode: "",
  });
  const [errors, setErrors] = useState({});

  const totalPrice = cartItems.reduce(
    (sum, item) => sum + item.price * item.quantity,
    0
  );

  const handleChange = (e) => {
    const { name, value } = e.target;
    setForm((prev) => ({ ...prev, [name]: value }));
  };

  const validate = () => {
```

```
  const newErrors = {};
  if (!form.name.trim()) newErrors.name = "Full name is required.";
  if (!form.email.trim()) {
    newErrors.email = "Email is required.";
  } else if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(form.email)) {
    newErrors.email = "Enter a valid email address.";
  }
  if (!form.address.trim()) newErrors.address = "Address is required.";
  if (!form.city.trim()) newErrors.city = "City is required.";
  if (!form.postalCode.trim())
    newErrors.postalCode = "Postal code is required.";
  return newErrors;
};

const handleSubmit = (e) => {
  e.preventDefault();

  const validationErrors = validate();
  if (Object.keys(validationErrors).length > 0) {
    setErrors(validationErrors);
    return;
  }

  // Read the total and build an order number BEFORE clearing the cart --
  // once clearCart() dispatches, cartItems (and therefore totalPrice) is gone.
  const order = {
    orderNumber: `ORD-${Date.now().toString(36).toUpperCase()}`,
    name: form.name.trim(),
    total: totalPrice,
  };

  dispatch(clearCart());
  onOrderPlaced(order);
};

if (cartItems.length === 0) {
  return (
    <div className="checkout-empty">
      <p>Your cart is empty -- add something before checking out.</p>
      <button className="back-btn" onClick={onBackToCart}>
        Back to Cart
      </button>
    </div>
  );
}

return (
  <div className="checkout">
    <div className="checkout-summary">
      <h3>Order Summary</h3>
      {cartItems.map((item) => (
        <div className="summary-row" key={item.id}>
          <span>
            {item.name} x {item.quantity}
          </span>
          <span>${(item.price * item.quantity).toFixed(2)}</span>
        </div>
      ))}
      <div className="summary-total">
        <span>Total</span>
        <span>${totalPrice.toFixed(2)}</span>
      </div>
    </div>

    <form className="checkout-form" onSubmit={handleSubmit} noValidate>
```

```
        <h3>Shipping Details</h3>

        <div className="form-group">
          <label htmlFor="name">Full Name</label>
          <input
            id="name"
            name="name"
            type="text"
            value={form.name}
            onChange={handleChange}
            className={errors.name ? "has-error" : ""}
          />
          {errors.name && <p className="field-error">{errors.name}</p>}
        </div>

        <div className="form-group">
          <label htmlFor="email">Email</label>
          <input
            id="email"
            name="email"
            type="email"
            value={form.email}
            onChange={handleChange}
            className={errors.email ? "has-error" : ""}
          />
          {errors.email && <p className="field-error">{errors.email}</p>}
        </div>

        <div className="form-group">
          <label htmlFor="address">Address</label>
          <input
            id="address"
            name="address"
            type="text"
            value={form.address}
            onChange={handleChange}
            className={errors.address ? "has-error" : ""}
          />
          {errors.address && <p className="field-error">{errors.address}</p>}
        </div>

        <div className="form-row">
          <div className="form-group">
            <label htmlFor="city">City</label>
            <input
              id="city"
              name="city"
              type="text"
              value={form.city}
              onChange={handleChange}
              className={errors.city ? "has-error" : ""}
            />
            {errors.city && <p className="field-error">{errors.city}</p>}
          </div>

          <div className="form-group">
            <label htmlFor="postalCode">Postal Code</label>
            <input
              id="postalCode"
              name="postalCode"
              type="text"
              value={form.postalCode}
              onChange={handleChange}
              className={errors.postalCode ? "has-error" : ""}
            />
```

```
            {errors.postalCode && (
              <p className="field-error">{errors.postalCode}</p>
            )}
          </div>
        </div>

        <div className="checkout-actions">
          <button type="button" className="back-btn" onClick={onBackToCart}>
            Back to Cart
          </button>
          <button type="submit" className="place-order-btn">
            Place Order (${totalPrice.toFixed(2)})
          </button>
        </div>
      </form>
    </div>
  );
}

export default Checkout;
```

Why compute `order` before dispatching `clearCart()`? `cartItems` (and therefore `totalPrice`) comes from `useSelector`, which reads live from the store. The instant `clearCart()` dispatches, `state.cart.items` becomes `[]` -- so the order number and total have to be captured into a plain object *first*, then handed up to `App` via `onOrderPlaced`, which is the only reason the confirmation screen can still show a total after the cart is gone.

Why does the empty-cart guard exist at all, given the "Proceed to Checkout" button only appears when the cart has items? It's a defensive branch for a state the normal flow can't reach through the UI, but a user could reach by refreshing mid-checkout, using the browser back/forward buttons, or in a future version of the app with URL-based routing. Handling it explicitly means the component never crashes trying to render an order summary for zero items.

3. The order confirmation screen

`src/components/OrderConfirmation.jsx`

```
// Shown once after Checkout clears the cart -- order details are passed
// down as props since the cart itself (the source of that data) is already gone.
function OrderConfirmation({ order, onContinueShopping }) {
  return (
    <div className="order-confirmation">
      <div className="confirmation-icon">OK</div>
      <h2>Thank you, {order.name}!</h2>
      <p>Your order has been placed successfully.</p>
      <p className="order-number">
        Order Number: <strong>{order.orderNumber}</strong>
      </p>
      <p className="order-total">Total Paid: ${order.total.toFixed(2)}</p>
      <button className="add-btn" onClick={onContinueShopping}>
        Continue Shopping
      </button>
    </div>
  );
}

export default OrderConfirmation;
```

4. Add a "Proceed to Checkout" button to the cart

src/components/Cart.jsx (updated -- accepts `onCheckout`, rest unchanged from Part 5)

```
import { useSelector, useDispatch } from "react-redux";
import {
 removeFromCart,
 increaseQuantity,
 decreaseQuantity,
} from "../features/cartSlice";

function Cart({ onCheckout }) {
 const cartItems = useSelector((state) => state.cart.items);
 const dispatch = useDispatch();

 const handleRemove = (productId) => dispatch(removeFromCart(productId));
 const handleIncrease = (productId) => dispatch(increaseQuantity(productId));
 const handleDecrease = (productId) => dispatch(decreaseQuantity(productId));

 const totalPrice = cartItems.reduce(
   (sum, item) => sum + item.price * item.quantity,
   0
 );

 if (cartItems.length === 0) {
   return <p className="empty-cart">Your cart is empty.</p>;
 }

 return (
   <div className="cart">
     {cartItems.map((item) => (
       <div className="cart-item" key={item.id}>
         <img src={item.image} alt={item.name} />
         <div className="cart-item-info">
           <h3>{item.name}</h3>
           <p>${item.price.toFixed(2)} each</p>

           <div className="quantity-controls">
             <button className="qty-btn" onClick={() => handleDecrease(item.id)}>-</button>
             <span className="qty-value">{item.quantity}</span>
             <button className="qty-btn" onClick={() => handleIncrease(item.id)}>+</button>
           </div>

           <p className="subtotal">
             Subtotal: ${(item.price * item.quantity).toFixed(2)}
           </p>
         </div>
         <button className="remove-btn" onClick={() => handleRemove(item.id)}>
           Remove
         </button>
       </div>
     ))}

     <div className="cart-total">
       <h3>Total: ${totalPrice.toFixed(2)}</h3>
       <button className="checkout-btn" onClick={onCheckout}>
         Proceed to Checkout
       </button>
     </div>
   </div>
 );
}

export default Cart;
```

5. Wire the four views together in App

`src/App.jsx` (updated)

```
import { useState } from "react";
import Navbar from "./components/Navbar";
import ProductList from "./components/ProductList";
import Cart from "./components/Cart";
import Checkout from "./components/Checkout";
import OrderConfirmation from "./components/OrderConfirmation";
import "./App.css";

function App() {
 // Tracks which view is showing: products, cart, checkout, or the
 // post-order confirmation. Still local UI state, not Redux -- same
 // reasoning as Part 4, just with two more possible values now.
 const [view, setView] = useState("products");

 // The most recently placed order, captured right before Checkout clears
 // the cart -- OrderConfirmation reads from this, not from Redux, since
 // by the time it renders the cart itself is already empty.
 const [lastOrder, setLastOrder] = useState(null);

 const handleOrderPlaced = (order) => {
   setLastOrder(order);
   setView("confirmation");
 };

 const handleContinueShopping = () => {
   setLastOrder(null);
   setView("products");
 };

 return (
   <div className="app">
     <Navbar onCartClick={() => setView("cart")} />
     <main>
       {view === "products" && (
         <>
           <h2>Products</h2>
           <ProductList />
         </>
       )}

       {view === "cart" && (
         <>
           <div className="cart-header">
             <h2>Your Cart</h2>
             <button className="back-btn" onClick={() => setView("products")}>
               Back to Products
             </button>
           </div>
           <Cart onCheckout={() => setView("checkout")} />
         </>
       )}

       {view === "checkout" && (
         <>
           <div className="cart-header">
             <h2>Checkout</h2>
             <button className="back-btn" onClick={() => setView("products")}>
               Back to Products
             </button>
           </div>
           <Checkout
             onOrderPlaced={handleOrderPlaced}
             onBackToCart={() => setView("cart")}
```

```
          />
        </>
      )}

      {view === "confirmation" && lastOrder && (
        <OrderConfirmation
          order={lastOrder}
          onContinueShopping={handleContinueShopping}
        />
      )}
    </main>
  </div>
 );
}

export default App;
```

Why `view === "confirmation" && lastOrder`, not just `view === "confirmation"`? `OrderConfirmation` reads `order.name` and `order.total` directly off its `order` prop with no fallback -- if `view` were somehow `"confirmation"` while `lastOrder` was still `null` (impossible through the UI as written, but cheap to guard against), rendering would throw. The `&&` guard means that branch simply renders nothing rather than crashing in that edge case.

6. Styling

`src/App.css` (append)

```
.cart-total {
 align-self: flex-end;
 margin-top: 8px;
 padding: 16px 24px;
 background: white;
 border-radius: 8px;
 box-shadow: 0 2px 6px rgba(0, 0, 0, 0.1);
 display: flex;
 align-items: center;
 gap: 20px;
}

.checkout-btn {
 background-color: #28a745;
 color: white;
 border: none;
 padding: 10px 20px;
 border-radius: 4px;
 cursor: pointer;
 font-size: 1em;
 font-weight: bold;
 transition: background-color 0.15s ease-in-out;
}
.checkout-btn:hover { background-color: #218838; }

/* ---------- Checkout page ---------- */

.checkout-empty { margin-top: 24px; color: #666; }

.checkout {
 display: grid;
 grid-template-columns: 1fr 1.4fr;
 gap: 24px;
 margin-top: 16px;
 align-items: start;
```

```
}

.checkout-summary {
 background: white;
 border-radius: 8px;
 padding: 20px;
 box-shadow: 0 2px 6px rgba(0, 0, 0, 0.1);
}
.checkout-summary h3 { margin-bottom: 12px; }

.summary-row {
 display: flex;
 justify-content: space-between;
 padding: 8px 0;
 border-bottom: 1px solid #eee;
 font-size: 0.95em;
 color: #444;
}

.summary-total {
 display: flex;
 justify-content: space-between;
 padding-top: 14px;
 margin-top: 6px;
 font-weight: bold;
 font-size: 1.1em;
}

.checkout-form {
 background: white;
 border-radius: 8px;
 padding: 20px;
 box-shadow: 0 2px 6px rgba(0, 0, 0, 0.1);
}
.checkout-form h3 { margin-bottom: 16px; }

.checkout-form .form-group {
 margin-bottom: 14px;
 display: flex;
 flex-direction: column;
 gap: 6px;
}

.checkout-form label { font-size: 0.85em; font-weight: bold; color: #444; }

.checkout-form input {
 padding: 10px 12px;
 border: 1px solid #ccc;
 border-radius: 6px;
 font-size: 1em;
}
.checkout-form input:focus { outline: none; border-color: #282c34; }
.checkout-form input.has-error { border-color: #d9534f; }

.field-error { color: #d9534f; font-size: 0.8em; margin: 0; }

.form-row {
 display: grid;
 grid-template-columns: 1fr 1fr;
 gap: 14px;
}

.checkout-actions {
 display: flex;
 justify-content: space-between;
```

```
 align-items: center;
 margin-top: 20px;
}

.place-order-btn {
 background-color: #28a745;
 color: white;
 border: none;
 padding: 10px 20px;
 border-radius: 4px;
 cursor: pointer;
 font-size: 1em;
 font-weight: bold;
 transition: background-color 0.15s ease-in-out;
}
.place-order-btn:hover { background-color: #218838; }

/* ---------- Order confirmation ---------- */

.order-confirmation {
 max-width: 420px;
 margin: 40px auto;
 background: white;
 border-radius: 8px;
 padding: 40px 32px;
 text-align: center;
 box-shadow: 0 2px 6px rgba(0, 0, 0, 0.1);
}

.confirmation-icon {
 width: 56px;
 height: 56px;
 line-height: 56px;
 border-radius: 50%;
 background-color: #28a745;
 color: white;
 font-size: 1.6em;
 margin: 0 auto 16px;
}

.order-confirmation h2 { margin-bottom: 8px; }
.order-confirmation p { color: #555; margin: 4px 0; }
.order-number { margin-top: 16px !important; }
.order-total { font-weight: bold; color: #222 !important; margin-bottom: 20px !important; }
```

`src/App.css` (append -- mobile breakpoints for checkout, inside the existing `@media (max-width: 600px)` block from Part 6)

```
.cart-total {
 align-self: stretch;
 text-align: right;
 flex-direction: column;
 align-items: stretch;
}

.checkout {
 grid-template-columns: 1fr;
}

.form-row {
 grid-template-columns: 1fr;
}

.checkout-actions {
```

```
 flex-direction: column-reverse;
 gap: 10px;
 align-items: stretch;
}

.order-confirmation {
 margin: 24px 16px;
 padding: 32px 20px;
}
```

7. Try it

```
npm run dev
```

Add items, open the cart, click **Proceed to Checkout**. Try submitting the form empty first -- you should see five field-level error messages and the cart should *not* clear. Fill in all five fields and submit: you land on a confirmation screen with a generated order number and the correct total, the cart badge in the navbar drops to 0, and (thanks to Part 8) `localStorage`'s `redux-cart-app/cart` entry is back down to `{ "items": [] }` too, since `clearCart()` is just another cart-changing dispatch that the Part 8 subscriber picks up like any other.

Command Reference

Command	Purpose
`npm create vite@latest redux-cart-app -- --template react`	Scaffolds the project: base folder structure, `index.html`, starter files.
`npm install @reduxjs/toolkit react-redux`	Store setup (`configureStore`, `createSlice`) and the React bindings (`Provider`, `useSelector`, `useDispatch`).
`npm install axios`	HTTP client used inside `productsSlice.js`'s `createAsyncThunk`.
`npm install`	Installs everything already listed in `package.json` -- the only command needed after a fresh clone.
`npm run dev`	Starts the Vite dev server with hot-module reloading.
`npm run build`	Bundles the app into `dist/` -- minified, production-ready static files.
`npm run preview`	Serves the `dist/` build locally, to sanity-check it before deploying.

Recap

- **Part 1** -- Vite + React scaffold, static product catalog, dumb display components, nothing wired up yet.
- **Part 2** -- `cartSlice` + `configureStore` + `<Provider>`. Store exists; nothing dispatches into it yet.
- **Part 3** -- Add to Cart dispatches `addToCart`; navbar count reads live from the store.

- **Part 4** -- A real cart view with Remove, toggled via local `useState`, not Redux.
- **Part 5** -- +/- quantity controls, per-item subtotals, a running total -- all derived, none stored.
- **Part 6** -- Responsive breakpoints, hover/transition polish, and the reasoning you'd give in an interview.
- **Part 7** -- Static catalog replaced by a live API fetch via `createAsyncThunk` + axios, with loading/error states.
- **Part 8** -- The cart survives a page reload: `preloadedState` rehydrates it from `localStorage` on startup, and a debounced `store.subscribe()` writes it back after every change.
- **Part 9** -- A full checkout flow: order summary, a validated shipping form, `clearCart()` on submit, and an order confirmation screen -- `App.jsx` now routes across four views instead of two.

That's the whole build -- clone the structure top to bottom and you'll land exactly where the working project already is.